CS315 · ALGORITHMS ANALYSIS & DESIGN

Lecture 1 Analysis of Algorithms

ملخص الميد · Goodrich Chapter 1

محارب

محتوى الملف

01 الفكرة

02 وقت التشغيل: Running Time

03 تجريبي ولا نظري؟

04 Pseudocode و RAM Model

05 Primitive Operations

06 عدّ العمليات في arrayMax

07 تقدير الوقت والـ Growth Rate

08 Big-Oh

09 قواعد الـ Big-Oh

10 Prefix Averages

11 قواعد Log و Exponent

12 أقارب الـ Big-Oh

13 ورقة المراجعة

01 الفكرة

المحاضرة تجاوب على سؤال واحد: كيف نقيس سرعة خوارزمية بدون ما نشغّلها على جهاز؟ الجواب: نعدّ العمليات البسيطة اللي تسويها، ونكتب العدد كدالة في حجم المدخل `n` ، وبعدين نبسّطها بالـ **Big-Oh**.

An algorithm is a step-by-step procedure for solving a problem in a finite amount of time.

وش يجي منها في الميد

سؤال ثابت: عدّ الـ **primitive operations** لخوارزمية **arrayMax**. وجملتين True/False تتكرر بالنص عن التحليل النظري والتجريبي.

02 وقت التشغيل: Running Time

- وقت التشغيل يكبر مع حجم المدخل.
- الـ **average case** صعب نحدده.
- نركّز على الـ **worst case**: أسهل في التحليل، ومهم في تطبيقات مثل الألعاب والمالية والروبوتات.

الحالة	معناها	مثال المحاضرة
Best case	أقل عدد عمليات.	**Insertion sort** والأرقام مرتبة أصلاً: `O(n)` .
Average case	المتوسط على كل المدخلات.	صعب نحدده.
Worst case	أكبر عدد عمليات. هذا الـ upper bound.	**Insertion sort** والأرقام معكوسة: `O(n²)` . و **Linear search** لما العنصر مو موجود.

03 تجريبي ولا نظري؟

Experimental Studies 1

1. Write a program implementing the algorithm.
2. Run the program with inputs of varying size and composition.
3. Use a method like System.currentTimeMillis() to measure the actual running time.
4. Plot the results.

Theoretical Analysis 2

1. Uses a high-level description of the algorithm instead of an implementation.
2. Characterizes running time as a function of the input size, n.
3. Takes into account all possible inputs.
4. Evaluates the speed of an algorithm independent of the hardware/software environment.

عيوب الطريقة التجريبية (**Limitations of Experiments**) ثلاثة:

1. It is necessary to implement the algorithm, which may be difficult.
2. Results may not be indicative of the running time on other inputs not included in the experiment.
3. In order to compare two algorithms, the same hardware and software environments must be used **and the same inputs**.

جملتين جو في الميد بالنص

In order to compare two algorithms, the same hardware and software environments must be used **without any consideration of inputs type and its size**.

False. لازم نفس المدخلات بعد.

Theoretical analysis allows us to evaluate the speed of an algorithm independent of the hardware/software environment.

True. هذي النقطة الرابعة من مميزات التحليل النظري.

04 Pseudocode و RAM Model

الـ **Pseudocode** وصف عالي المستوى للخوارزمية: مرتب أكثر من الكلام العادي، وأقل تفصيل من البرنامج، ويخفي تفاصيل تصميم البرنامج.

```
Algorithm arrayMax(A, n)
    Input   array A of n integers
    Output  maximum element of A
    currentMax ← A[0]
    for i ← 1 to n − 1 do
        if A[i] > currentMax then
            currentMax ← A[i]
    return currentMax
```

الـ **RAM Model** (Random Access Machine) هو الجهاز الافتراضي اللي نحلل عليه:

- فيه CPU واحد.
- ذاكرة خلاياها مرقمة وما لها حد، وكل خلية تشيل رقم أو حرف.

- الوصول لأي خلية ياخذ **unit time**، يعني وقت ثابت.

05 Primitive Operations

هي العمليات الأساسية اللي تسويها الخوارزمية. نشوفها في الـ pseudocode، وما تعتمد على لغة البرمجة، ونفترض كل وحدة تاخذ وقت ثابت في الـ RAM model.

العملية	مثال	العدد
Evaluating an expression	`i + 1` , `A[i] > currentMax`	1
Assigning a value to a variable	`currentMax ← …`	1
Indexing into an array	`A[i]`	1
Calling a method	`arrayMax(A, n)`	1
Returning from a method	`return currentMax`	1

06 عدّ العمليات في arrayMax

هذا سؤال الميد. احفظ الجدول وافهم من وين جا كل رقم:

السطر	# operations	ليش
`currentMax ← A[0]`	`2`	indexing + assignment.
`for i ← 1 to n − 1 do`	`1 + n`	`i ← 1` مرة وحدة، والمقارنة `i ≤ n − 1` تنحسب `n` مرة (آخر مرة تفشل وتطلع من اللوب).
`if A[i] > currentMax then`	`2(n − 1)`	indexing + مقارنة، في كل لفة.
`currentMax ← A[i]`	`2(n − 1)`	indexing + assignment. في الـ worst case تتنفذ كل لفة.
`{ increment counter i }`	`2(n − 1)`	جمع + assignment.
`return currentMax`	`1`	return.
Total	**`7n − 2`**	اللوب يلف `n − 1` مرة.

```
Worst case:   2 + (1 + n) + 2(n − 1) + 2(n − 1) + 2(n − 1) + 1  =  7n − 2
```

```
Best case:    2 + (1 + n) + 2(n − 1) +      0     + 2(n − 1) + 1  =  5n
```

الـ worst case لما المصفوفة مرتبة تصاعدي، فكل عنصر أكبر من اللي قبله. والـ best case لما أكبر عنصر هو `A[0]`، فالسطر الرابع ما يتنفذ أبداً.

نسخة الميد

في ميد 451 و 442 زاد الدكتور سطر `Temp ← A[0]` بعد أول سطر. هذا يزيد عمليتين، فالجواب يصير `7n − 2 + 2 = 7n` .

ملاحظة: سلايد 23 يكتب «at most 7n − 1»، وسلايد 12 و 14 يكتبون `7n − 2` . الصحيح حسب الجدول `7n − 2` .

07 تقدير الوقت والـ Growth Rate

```
a = time taken by the fastest primitive operation
```

```
b = time taken by the slowest primitive operation
```

```
a(7n − 2)  ≤  T(n)  ≤  b(7n − 2)
```

يعني `T(n)` محصورة بين دالتين خطيتين، فهي خطية.

- تغيير الـ hardware أو الـ software يأثر على `T(n)` بـ **constant factor** بس.
- لكنه ما يغيّر الـ **growth rate**.
- الـ growth rate ما يتأثر بالـ constant factors ولا بالـ **lower-order terms**.

الدالة	نوعها
$10^2n + 10^5$	Linear
$10^5n^2 + 10^8n$	Quadratic

ترتيب الدوال من الأبطأ نمواً إلى الأسرع:

```
1  <  log n  <  n  <  n log n  <  n²  <  n³  <  2ⁿ
```

Big-Oh 08

Given functions f(n) and g(n), we say that f(n) is O(g(n)) if there are positive constants c and n_0 such that

$f(n) \leq c \cdot g(n)$ for $n \geq n_0$

بالعربي: `g(n)` مضروبة في ثابت تصير فوق `f(n)` من نقطة معينة `n₀` وطالع. الـ Big-Oh يعطي **upper bound**.

Ex **أثبت أن `O(n)` is `2n + 10`**

```
2n + 10 ≤ cn
```

```
(c − 2)n ≥ 10
```

```
n ≥ 10 / (c − 2)
```

```
Pick c = 3 and n₀ = 10
```

Ex **أثبت أن `O(n)` is not `n²`**

```
n² ≤ cn
```

```
n ≤ c
```

مستحيل، لأن `c` لازم يكون ثابت و `n` تكبر بلا حد.

n_0	c	Big-Oh	f(n)
1	7	$O(n)$	$7n - 2$
21	4	$O(n^3)$	$3n^3 + 20n^2 + 5$
2	4	$O(\log n)$	$3 \log n + \log \log n$

طريقة سريعة تطلّع c و n_0

كبّر كل حد إلى أعلى أس. مثال: $3n^3 + 20n^2 + 5 \leq 3n^3 + 20n^3 + 5n^3 = 28n^3$ لكل $n \geq 1$، فـ $c = 28$ و $n_0 = 1$. أي زوج يحقق المتراجحة صحيح، ما فيه جواب واحد.

09 قواعد الـ Big-Oh

- لو $f(n)$ polynomial درجته d فهو $O(n^d)$: احذف الـ lower-order terms واحذف الـ constant factors.
- استخدم أصغر class ممكن: قل $2n$ is $O(n)$ مو $O(n^2)$.
- استخدم أبسط شكل: قل $3n + 5$ is $O(n)$ مو $O(3n)$.

	f(n) is O(g(n))	g(n) is O(f(n))
g(n) grows more	Yes	No
f(n) grows more	No	Yes
Same growth	Yes	Yes

الـ **Asymptotic analysis** خطوتين: نطلّع عدد العمليات في الـ worst case كدالة في n، وبعدين نكتبها بالـ Big-Oh. مثال: **arrayMax** تنفذ $7n - 2$ عملية، فنقول «runs in $O(n)$ time».

Prefix Averages 10

المطلوب مصفوفة `A` فيها `A[i]` = متوسط أول `i + 1` عنصر من `X`:

```
A[i] = (X[0] + X[1] + … + X[i]) / (i + 1)
```

Quadratic · O(n²) 1

```
prefixAverages1(X, n)          # ops
A ← new array of n integers     n
for i ← 0 to n − 1 do           n
   s ← X[0]                     n
   for j ← 1 to i do            1+2+…+(n−1)
      s ← s + X[j]              1+2+…+(n−1)
   A[i] ← s / (i + 1)           n
return A                        1
```

Linear · O(n) 2

```
prefixAverages2(X, n)          # ops
A ← new array of n integers     n
s ← 0                           1
for i ← 0 to n − 1 do           n
   s ← s + X[i]                 n
   A[i] ← s / (i + 1)           n
return A                        1
```

الأولى تعيد الجمع من البداية كل مرة، فاللوب الداخلي يلف `1 + 2 + … + (n − 1)` مرة. الثانية تحتفظ بمجموع جاري `s` (**running sum**) فتكفيها لوب وحدة.

```
Arithmetic progression:   1 + 2 + … + n  =  n(n + 1) / 2     →   O(n²)
```

11 قواعد Log و Exponent

```
log_b(xy)   = log_b x + log_b y
```

```
log_b(x/y) = log_b x − log_b y
```

```
log_b(xᵃ)   = a · log_b x
```

```
log_b a     = log_x a / log_x b
```

```
a^(b+c)   = a^b · a^c
```

```
a^(bc)    = (a^b)^c
```

```
a^b / a^c = a^(b−c)
```

```
b = a^(log_a b)
```

12 أقارب الـ Big-Oh

الرمز	الشرط	المعنى
Big-Oh O	$f(n) \le c \cdot g(n)$	`f` أقل من أو تساوي `g` (upper bound).
Big-Omega Ω	$f(n) \ge c \cdot g(n)$	`f` أكبر من أو تساوي `g` (lower bound).
Big-Theta Θ	$c' \cdot g(n) \le f(n) \le c'' \cdot g(n)$	`f` تساوي `g` في النمو.
little-oh o	$f(n) \le c \cdot g(n)$ for any $c > 0$	`f` أقل تماماً من `g`.
little-omega ω	$f(n) \ge c \cdot g(n)$ for any $c > 0$	`f` أكبر تماماً من `g`.

الفرق بين الكبير والصغير: في O و Ω يكفي ثابت واحد `c` يحقق الشرط. في o و ω الشرط لازم يتحقق لأي `c > 0`.

13 ورقة المراجعة

السؤال	الجواب
على وش نركّز في التحليل؟	الـ worst case.
عمليات **arrayMax**؟	`7n - 2` في الـ worst case و `5n` في الـ best. ومع سطر Temp: `7n`.
تعريف Big-Oh؟	$f(n) \leq c \cdot g(n)$ for $n \geq n_0$
وش يتأثر بتغيير الجهاز؟	الـ constant factor فقط، مو الـ growth rate.
prefixAverages1 / 2	$O(n^2)$ / $O(n)$
$1 + 2 + \ldots + n$	$n(n + 1) / 2$